System Design Basics

Lesson 5: Scalability & Performance

Scalability & Performance - Study Notes

Key Concepts

1. **Load Balancing** – Distributing incoming traffic across multiple servers to avoid overloading any single instance.

2. **Caching** – Storing frequently accessed data in fast, temporary storage (memory, SSD, etc.) to reduce latency and database load.

3. **Content Delivery Network (CDN)** – A geographically distributed network of servers that caches static assets close to end users, lowering round trip time.

4. **Horizontal vs. Vertical Scaling** – Adding more machines (horizontal) vs. upgrading existing hardware (vertical) to handle increased load.

5. **Cache Invalidation & Consistency** – Strategies to keep cached data fresh and in sync with the source of truth.

Summary

Scalability and performance are critical for building applications that can grow without compromising user experience. Load balancing ensures that no single server becomes a bottleneck by spreading requests across a pool of instances. Caching reduces the number of expensive database hits by keeping hot data in fast memory, while CDNs push static content closer to users, cutting latency and bandwidth usage. Together, these techniques allow systems to handle higher traffic, lower response times, and more reliable uptime.

A well designed architecture will combine these layers: a load balancer front ends a cluster of stateless application servers, each backed by a shared cache (e.g., Redis) for session data and frequently accessed queries, while static assets are served from a CDN. Proper cache invalidation, health checks, and monitoring complete the loop, ensuring that performance gains are sustainable and consistent.

Important Points

- **Statelessness**: Keep application servers stateless so any instance can handle any request; use external stores for state (sessions, queues).
- **Health Checks**: Load balancers should regularly probe instances and remove unhealthy nodes from rotation.
- **Cache Granularity**: Cache at the right level (query, object, page) to balance hit rate vs. memory cost.
- **Expiration Policies**: Use TTLs, LRU, or explicit invalidation to keep cache data fresh.
- **CDN Caching Headers**: Set `Cache-Control`, `ETag`, and `Expires` headers to control CDN behavior.
- **Monitoring**: Track cache hit ratios, latency, and error rates to detect performance regressions early.

- **Security**: Encrypt data in transit (TLS) and consider encryption at rest for sensitive cached data.

Examples

1. Redis Cache for Session Data

```python
# Flask example

from flask import Flask, session
from redis import Redis
from flask_session import Session
app = Flask(__name__)
app.config['SESSION_TYPE'] = 'redis'
app.config['SESSION_REDIS'] = Redis(host='redis', port=6379)
Session(app)
@app.route('/')
def index():
    session['visits'] = session.get('visits', 0) + 1
    return f"Visits: {session['visits']}"
```

Explanation: The `flask_session` extension stores session cookies in Redis, allowing any server behind a load balancer to read/write session data. This removes the need for sticky sessions and scales horizontally.

2. CDN for Static Assets

```html
<link rel="stylesheet" href="https://cdn.example.com/css/main.min.css">
<script src="https://cdn.example.com/js/app.bundle.js"></script>
```

Explanation: By serving CSS and JavaScript from a CDN, the browser fetches these assets from a server geographically close to the user, reducing latency and offloading traffic from the origin server.

Quick Review

1. What is the primary benefit of using a load balancer in a web application?
2. How does caching improve database performance, and what is a common cache invalidation strategy?
3. Explain the difference between horizontal and vertical scaling.
4. Why is it important for application servers to be stateless when using load balancing?
5. What HTTP header can you set to instruct a CDN to cache a resource for 24 hours?

Further Reading

- **CAP Theorem** – Understanding consistency, availability, and partition tolerance trade offs.

• **Microservices Architecture** – How stateless services and shared caches fit into a microservice ecosystem.

• **Database Sharding & Partitioning** – Scaling data storage horizontally.

• **Observability & APM** – Tools for monitoring latency, error rates, and cache metrics.

• **Edge Computing** – Extending CDN concepts to compute at the network edge.